

Angular DevExtreme Rapid Study Plan

<https://gemini.google.com/app/95cf5be063ffccff>

User prompt: frontend files are missing

Response: If the frontend files are missing from your GitHub repository after pushing, it usually means your **.gitignore file in the root directory is accidentally ignoring the frontend folder**, or the frontend folder was added as a **nested Git repository (git submodule)**.

Here is how to check and fix both issues.

Cause 1: Nesting Git Repositories (Most Common)

If your Angular project was created using `ng new frontend`, Angular automatically initializes its own `.git` folder inside that subfolder. When you run `git add .` from the root, Git ignores nested `.git` folders instead of tracking their contents.

Fix:

1. **Delete the hidden `.git` folder inside your frontend directory:**

- **Windows (PowerShell):**

```
Remove-Item -Recurse -Force .\frontend\.git
```

- **Mac / Linux:**

```
rm -rf frontend/.git
```

2. **Clear Git's index cache for the frontend folder:**

```
git rm --cached -r frontend
```

3. **Stage and commit all frontend files:**

```
git add .  
git commit -m "fix: include frontend project files"  
git push
```

Cause 2: Overly Broad `.gitignore` File

If your root `.gitignore` contains a generic pattern like `dist/`, `build/`, or an unspecific folder match, Git might be ignoring your entire frontend directory.

Fix:

1. Open your root .gitignore file.
2. Ensure you are ignoring build outputs specifically inside subdirectories rather than ignoring whole folder trees:

```
# Correct specific ignores
frontend/dist/
frontend/.angular/
frontend/node_modules/
```

3. Test if Git recognizes the files:
git status

(You should now see all Angular files listed under "Untracked files" or "Changes to be committed".)

Cause 3: Checking Tracked Files

To verify what Git is tracking locally before pushing:

```
git ls-files frontend/
```

If this command outputs empty, repeat **Cause 1** steps above. Once you see your .ts, .html, and package.json files listed, push to GitHub:

```
git add .
git commit -m "docs: add frontend codebase"
git push
```

*User prompt: # Build outputs bin/ obj/ # User-specific files *.user *.userosscache *.suo .vs/ appsettings.Development.json this was in frontend gitignore # Node & Angular frontend/node_modules/ frontend/.angular/ frontend/dist/ frontend/.vscode/ frontend/.idea/ # Environment local overrides frontend/.env*

Response: Your current split setup works, but having bin/, obj/, .vs/, and *.user inside the **frontend** .gitignore doesn't do anything because those are C# / .NET build artifacts.

For a single root repository containing both frontend/ and backend/ folders, keep all ignore rules in **one .gitignore file located at the very root** of your repository.

Recommended Single Root .gitignore

Replace your root .gitignore file with this unified version:

```
# =====
# Frontend (Angular & Node)
# =====
frontend/node_modules/
frontend/.angular/
frontend/dist/
```

```

frontend/.vscode/
frontend/.idea/
frontend/.env

# =====
# Backend (.NET / C#)
# =====
**/bin/
**/obj/
**/.vs/
*.user
*.userosscache
*.suo
appsettings.Development.json
appsettings.Production.json

# =====
# General System Files
# =====
.DS_Store
Thumbs.db

```

Step-by-Step Fix to Push Missing Frontend Files

If you previously tried to push and the frontend files were skipped, follow these terminal steps from your repository root folder:

1. Remove Any Sub-Git Folder in Frontend

If Angular initialized its own Git repository inside frontend/, Git will treat it as a submodule and skip its contents:

- **Windows (PowerShell):**
`Remove-Item -Recurse -Force .\frontend\.git`
- **Mac / Linux:**
`rm -rf frontend/.git`

2. Reset Git Cache and Stage Files

Tell Git to untrack the cached folder reference and re-scan everything:

```

# Clear cached sub-module tracking
git rm -r --cached frontend

# Stage all files according to the new root .gitignore
git add .

# Verify frontend files are now staged

```

```
git status
```

3. Commit and Push

Once you see your Angular .ts, .html, and package.json files listed under green staged changes:

```
git commit -m "fix: add full frontend codebase to repository"
git push
```

User prompt: frontend was created using git, so it was not committed in repo, pls do the needful to move the files into repo

Response: To convert the frontend sub-repository into a standard folder inside your root Git repository, run these commands in your project's root terminal:

1. Remove the Embedded .git Folder from frontend

This removes the independent Git repository tracking inside the Angular directory.

- **Windows (PowerShell):**
`Remove-Item -Recurse -Force .\frontend\.git`
- **Command Prompt (cmd):**
`rmdir /s /q frontend\.git`
- **Mac / Linux:**
`rm -rf frontend/.git`

2. Clear Git's Cache for the frontend Submodule

Clear the cached gitlink reference in the parent repository:

```
git rm --cached -r frontend
```

3. Stage, Commit, and Push the Frontend Files

Stage the entire folder so Git recognizes all Angular source files, then push to GitHub:

```
# Stage all files
git add frontend/
```

```
# Check status (you should see green untracked/added files under frontend/)
git status
```

```
# Commit and push
git commit -m "fix: integrate frontend source code into main repository"
git push
```

User prompt: provide me as in tortoise git, steps

Response: Here are the exact step-by-step instructions using **TortoiseGit** (Windows GUI) to move your frontend files into the main repository:

Step 1: Delete the Embedded .git Folder in frontend

Since the frontend directory was initialized as its own Git repository, its hidden .git folder must be deleted first.

1. Open **Windows File Explorer** and navigate to your frontend folder.
2. Ensure hidden items are visible: Click **View** → check **Hidden items**.
3. Locate the .git folder inside frontend and **Delete** it.

Step 2: Clear the Git Cache in TortoiseGit

1. Right-click inside your root project folder (where both frontend and backend reside).
2. Select **Git Commit -> "main"...**
3. Look at the file list at the bottom:
 - If frontend appears as a submodule/folder icon with a red or grey mark, right-click on frontend in the list → select **Delete (keep local)** or **Revert**.
4. Close the commit window.

Step 3: Add and Commit the Frontend Files

1. Right-click in the empty space of your root project folder.
2. Click **TortoiseGit** → **Add...**
3. In the dialog, check the box next to frontend (and make sure node_modules or .angular are unchecked).
4. Click **OK**.
5. Once added, click **Commit...** in the completion window (or right-click the folder and select **Git Commit -> "main"...**).
6. Enter a commit message (e.g., fix: add frontend source files to main repo).
7. Check all the files listed under frontend/ (such as src/, package.json, etc.) to make sure they are checked in green/blue.
8. Click **Commit**.

Step 4: Push to GitHub

1. Right-click inside the root project folder.
2. Select **TortoiseGit** → **Push...**
3. Ensure **Remote** is set to origin and **Branch** is main.
4. Click **OK**.

Once completed, refresh your GitHub repository page in the browser—your full frontend source code will

now be visible.

User prompt: these are the folders of frontend, why the frontend subfolders are not listed in git

Response: Looking at your folder structure, Git is excluding or skipping these specific folders for two distinct reasons:

Why Certain Folders Are Excluded

1. **.angular and node_modules (Ignored by .gitignore)**
 - These contain build caches and heavy third-party packages.
 - They are intentionally ignored by .gitignore so you don't upload hundreds of megabytes of unnecessary dependency files to GitHub.
2. **src, public, and .vscode (Skipped due to Stale Git Cache)**
 - Git previously registered Frontend as a single submodule/gitlink (because Angular created a .git folder inside it originally).
 - Even after deleting the hidden .git folder, the main Git repository still has an entry in its index telling it to treat Frontend as an embedded repository link rather than scanning its contents (src, public, etc.).

How to Fix Using TortoiseGit

Follow these exact steps in TortoiseGit to force Git to track all source subfolders (src, public, etc.):

Step 1: Open the Commit Window from Root

1. Go up one level to D:\MyFullStackProject.
2. Right-click inside MyFullStackProject → **Git Commit -> "main"...**

Step 2: Unstage/Delete the Cached Frontend Entry

1. Look at the bottom file list in the commit window.
2. Find the entry for Frontend (it will usually have a folder icon or grey/submodule indicator).
3. Right-click Frontend → select **Revert** or **Delete (keep local)** to remove the cached submodule index entry.
4. Close the commit window.

Step 3: Add Frontend Back to Stage

1. Right-click in D:\MyFullStackProject.
2. Select **TortoiseGit** → **Add...**
3. Expand the tree in the Add window: select **Frontend**.
4. Click **OK**.

Step 4: Commit and Push

1. Right-click in D:\MyFullStackProject → **Git Commit -> "main"...**
2. You will now see all individual subfolders and files (Frontend/src/..., Frontend/public/..., Frontend/package.json, etc.) listed in green/blue.

3. Type a commit message (e.g., fix: track frontend source files).
 4. Click **Commit**, then click **Push...** to upload everything to GitHub.
-

User prompt: new file: Frontend/.editorconfig new file: Frontend/.gitignore new file: Frontend/.prettierrc new file: Frontend/README.md new file: Frontend/angular.json new file: Frontend/package-lock.json new file: Frontend/package.json new file: Frontend/public/favicon.ico new file: Frontend/src/app/app.component.css new file: Frontend/src/app/app.component.html new file: Frontend/src/app/app.component.ts new file: Frontend/src/app/app.config.ts new file: Frontend/src/app/app.routes.ts new file: Frontend/src/app/app.spec.ts new file: Frontend/src/index.html new file: Frontend/src/main.ts new file: Frontend/src/styles.css new file: Frontend/tsconfig.app.json new file: Frontend/tsconfig.json new file: Frontend/tsconfig.spec.json does this file really need to push to repo

Response: Yes, almost all of these files are required for your Angular application to run, build, and maintain consistent code formatting across environments.

Files You Must Push

- **Frontend/src/... (app.component.ts, app.component.html, main.ts, etc.)**
 - **Essential:** Contains all your actual Angular application source code and template logic.
- **Frontend/package.json & Frontend/package-lock.json**
 - **Essential:** Declares project dependencies (devextreme, zone.js, Angular packages) and locks exact versions so the app installs reliably on other machines using npm install.
- **Frontend/angular.json**
 - **Essential:** Defines build settings, polyfills, asset paths, and project configuration for the Angular CLI.
- **Frontend/tsconfig.json, tsconfig.app.json, tsconfig.spec.json**
 - **Essential:** Configures the TypeScript compiler rules for building and testing the Angular app.
- **Frontend/public/favicon.ico & Frontend/src/index.html, styles.css**
 - **Essential:** Provides the root HTML template, icon, and global CSS styles for the frontend layout.
- **Frontend/.editorconfig & Frontend/.prettierrc**
 - **Recommended:** Maintains consistent indentation and formatting rules across different IDEs and contributors.
- **Frontend/README.md & Frontend/.gitignore**
 - **Recommended:** Documents frontend setup instructions and defines ignore rules for future builds.

File You Can Optionally Exclude

- **Frontend/src/app/app.spec.ts**
 - **Optional:** This is the default auto-generated unit test file. If you haven't written unit tests, you can keep it or delete/ignore it.

What to Do Next

Keep all these files checked and proceed to **Commit** and **Push** in TortoiseGit. Your repository will then contain the complete, runnable Angular frontend codebase alongside your backend API.

User prompt: how can i download from the repo

Response: To download and set up your project from a public GitHub repository, you have two simple methods depending on your workflow:

Method 1: Git Clone (Recommended)

Using git clone keeps the connection to the repository so you can pull future updates or push changes.

1. Open **Command Prompt**, **PowerShell**, or **Git Bash**.
2. Run the clone command using your repository URL:
`git clone https://github.com/YOUR_USERNAME/YOUR_REPOSITORY_NAME.git`
3. Navigate into the cloned directory:
`cd YOUR_REPOSITORY_NAME`

*(If using **TortoiseGit**: Right-click inside your desired download folder → click **Git Clone...** → paste your GitHub URL → click **OK**).*

Method 2: Download ZIP (Quick Download)

If you just want a raw copy of the project files without Git version control:

1. Open your repository page on [GitHub](#).
2. Click the green <> **Code** button near the top right.
3. Click **Download ZIP**.
4. Extract the .zip file on your local machine.

Steps to Run After Downloading

Once downloaded, restore dependencies for both parts of your application:

1. Set up the Frontend (Angular):

```
cd Frontend
npm install
ng serve
```

2. Set up the Backend (.NET API):

```
cd Backend
dotnet restore
dotnet run
```

User prompt: Likewise I need the another project, name : sales management system 1. Database schema - user master - item master - customer master - sales item 2. Application screen and UI design 2.1 Authentication and master - user login: jwt - master screens: use crud operations for customer and item master, item master should have upload image and preview option 2.2 sales invoice entry screen - navigation, top , bottom [Please use own knowledge to implement] 3. Dashboard and reporting 3.1 interactive sales report 3.2 graphical dashboard These above are shown in the assessment paper, I think I might forget something, so provide me step by step implementation to achieve this by technologies like devextreme, sql server database, c# etc like previous inventory management..

Response: Here is the end-to-end implementation plan and code structure for building the **Sales Management System** using **ASP.NET Core Web API, SQL Server, Entity Framework Core, and Angular with DevExtreme**.

1. Database Schema (SQL Server)

```
-- 1. Users Table
CREATE TABLE Users (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Username NVARCHAR(50) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    FullName NVARCHAR(100) NOT NULL,
    Role NVARCHAR(20) NOT NULL DEFAULT 'User'
);

-- 2. Items Table
CREATE TABLE Items (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    ItemCode NVARCHAR(20) NOT NULL UNIQUE,
    Name NVARCHAR(100) NOT NULL,
    Price DECIMAL(18,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    ImagePath NVARCHAR(255) NULL
);

-- 3. Customers Table
CREATE TABLE Customers (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    CustomerCode NVARCHAR(20) NOT NULL UNIQUE,
    Name NVARCHAR(100) NOT NULL,
    Email NVARCHAR(100) NULL,
    Phone NVARCHAR(20) NULL,
    Address NVARCHAR(250) NULL
);

-- 4. Sales Header & Items Tables
CREATE TABLE SalesInvoices (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    InvoiceNumber NVARCHAR(30) NOT NULL UNIQUE,
    InvoiceDate DATETIME NOT NULL DEFAULT GETDATE(),
    CustomerId INT NOT NULL,
```

```

        TotalAmount DECIMAL(18,2) NOT NULL,
        CONSTRAINT FK_SalesInvoices_Customers FOREIGN KEY (CustomerId) REFERENCES
Customers(Id)
    );

CREATE TABLE SalesInvoiceItems (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    SalesInvoiceId INT NOT NULL,
    ItemId INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(18,2) NOT NULL,
    SubTotal AS (Quantity * UnitPrice),
    CONSTRAINT FK_SalesItems_Invoices FOREIGN KEY (SalesInvoiceId) REFERENCES
SalesInvoices(Id) ON DELETE CASCADE,
    CONSTRAINT FK_SalesItems_Items FOREIGN KEY (ItemId) REFERENCES Items(Id)
);

```

2. Backend Implementation (C# / ASP.NET Core)

2.1 Entity Framework Core Models

```

public class Item
{
    public int Id { get; set; }
    public string ItemCode { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public int StockQuantity { get; set; }
    public string? ImagePath { get; set; }
}

public class SalesInvoice
{
    public int Id { get; set; }
    public string InvoiceNumber { get; set; } = string.Empty;
    public DateTime InvoiceDate { get; set; } = DateTime.UtcNow;
    public int CustomerId { get; set; }
    public Customer? Customer { get; set; }
    public decimal TotalAmount { get; set; }
    public List<SalesInvoiceItem> Items { get; set; } = new();
}

public class SalesInvoiceItem
{
    public int Id { get; set; }
    public int SalesInvoiceId { get; set; }
    public int ItemId { get; set; }
    public Item? Item { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
}

```

2.2 Image Upload Endpoint (ItemsController.cs)

```
[HttpPost("upload-image")]
public async Task<IActionResult> UploadImage(IFormFile file)
{
    if (file == null || file.Length == 0)
        return BadRequest("No image file provided.");

    var uploadsFolder = Path.Combine(Directory.GetCurrentDirectory(),
"wwwroot", "uploads");
    if (!Directory.Exists(uploadsFolder))
        Directory.CreateDirectory(uploadsFolder);

    var fileName = $"{Guid.NewGuid()} {Path.GetExtension(file.FileName)}";
    var filePath = Path.Combine(uploadsFolder, fileName);

    using (var stream = new FileStream(filePath, FileMode.Create))
    {
        await file.CopyToAsync(stream);
    }

    var relativePath = $"/uploads/{fileName}";
    return Ok(new { imagePath = relativePath });
}
```

2.3 Sales Invoice Save Endpoint (SalesController.cs)

```
[HttpPost]
public async Task<IActionResult> CreateInvoice([FromBody] SalesInvoice invoice)
{
    if (!ModelState.IsValid) return BadRequest(ModelState);

    // Auto-generate Invoice Number
    invoice.InvoiceNumber =
$"INV-{DateTime.UtcNow:yyyyMMdd}-{Guid.NewGuid().ToString()[..4].ToUpper()}";
    invoice.InvoiceDate = DateTime.UtcNow;

    _context.SalesInvoices.Add(invoice);
    await _context.SaveChangesAsync();

    return Ok(invoice);
}
```

3. Frontend UI Design (Angular & DevExtreme)

3.1 Item Master with Image Upload & Preview (item-master.component.html)

```
<dx-data-grid
```

```

[dataSource]="itemDataSource"
[showBorders]="true"
[remoteOperations]="true">

<dxo-editing mode="popup" [allowAdding]="true" [allowUpdating]="true"
[allowDeleting]="true">
  <dxo-popup title="Item Master" [showTitle]="true" [width]="700"
[height]="520"></dxo-popup>
  <dxo-form>
    <dxo-item dataField="itemCode"></dxo-item>
    <dxo-item dataField="name"></dxo-item>
    <dxo-item dataField="price"></dxo-item>
    <dxo-item dataField="stockQuantity"></dxo-item>
    <dxo-item dataField="imagePath"
template="imageUploadTemplate"></dxo-item>
  </dxo-form>
</dxo-editing>

<dxo-column dataField="itemCode" caption="Item Code"></dxo-column>
<dxo-column dataField="name" caption="Name"></dxo-column>
<dxo-column dataField="price" caption="Price"
format="$#,##0.00"></dxo-column>

<!-- Grid Preview Template -->
<dxo-column dataField="imagePath" caption="Preview"
cellTemplate="gridImageTemplate" [allowFiltering]="false">
</dxo-column>

<div *dxTemplate="let cell of 'gridImageTemplate'">
  <img [src]="cell.value" *ngIf="cell.value" style="height: 40px;
border-radius: 4px;" />
</div>

<!-- Popup Upload Form Template -->
<div *dxTemplate="let data of 'imageUploadTemplate'">
  <label>Item Image</label>
  <dx-file-uploader
    selectButtonText="Select Image"
    labelText="or Drop image here"
    accept="image/*"
    uploadMode="instantly"
    uploadUrl="http://localhost:7000/api/items/upload-image"
    (onUploaded)="onImageUploaded($event, data)">
  </dx-file-uploader>
  <div *ngIf="data.component.option('formData').imagePath" style="margin-top:
10px;">
    <img [src]="data.component.option('formData').imagePath" style="height:
80px; border-radius: 4px;" />
  </div>
</div>
</dx-data-grid>

```

3.2 Sales Invoice Entry Screen (sales-entry.component.html)

A Master-Detail transaction layout featuring customer selection at the top and line item grid in the center:

```
<div style="padding: 20px;">
  <h2>Sales Invoice Entry</h2>

  <!-- Header Form -->
  <dx-form [formData]="invoiceHeader" [colCount]="2">
    <dx-item dataField="customerId" editorType="dxSelectBox"
      [editorOptions]="{ dataSource: customersDataSource, displayExpr: 'name',
valueExpr: 'id', placeholder: 'Select Customer...' }">
      <dx-validation-rule type="required" message="Customer is
required"></dx-validation-rule>
    </dx-item>

    <dx-item dataField="invoiceDate" editorType="dxDateBox" [editorOptions]="{
value: currentDate, disabled: true }">
    </dx-item>
  </dx-form>

  <h3 style="margin-top: 20px;">Invoice Items</h3>

  <!-- Line Items Editing Grid -->
  <dx-data-grid
    [dataSource]="invoiceItems"
    [showBorders]="true"
    (onRowInserted)="calculateTotal()"
    (onRowUpdated)="calculateTotal()"
    (onRowRemoved)="calculateTotal()">

    <dxo-editing mode="cell" [allowAdding]="true" [allowUpdating]="true"
[allowDeleting]="true"></dxo-editing>

    <dx-column dataField="itemId" caption="Item" editorType="dxSelectBox"
      [editorOptions]="{ dataSource: itemsDataSource, displayExpr: 'name',
valueExpr: 'id' }">
    </dx-column>
    <dx-column dataField="unitPrice" caption="Unit Price" dataType="number"
format="$#,##0.00"></dx-column>
    <dx-column dataField="quantity" caption="Quantity"
dataType="number"></dx-column>
    <dx-column caption="SubTotal" [calculateCellValue]="calculateSubTotal"
format="$#,##0.00" [allowEditing]="false"></dx-column>
  </dx-data-grid>

  <!-- Summary & Save Bar -->
  <div style="margin-top: 20px; display: flex; justify-content: space-between;
align-items: center;">
```

```

    <h3>Total Amount: {{ totalAmount | currency }}</h3>
    <dx-button text="Save Invoice" type="success"
(onClick)="saveInvoice()"></dx-button>
  </div>
</div>

```

4. Dashboard & Reporting

4.1 Interactive Sales Report (sales-report.component.html)

Uses DevExtreme dx-pivot-grid for multi-dimensional slice-and-dice sales analysis:

```

<dx-pivot-grid
  [dataSource]="pivotDataSource"
  [allowSortingBySummary]="true"
  [allowFiltering]="true"
  [showBorders]="true">
  <dxo-field-chooser [enabled]="true"></dxo-field-chooser>
</dx-pivot-grid>

```

4.2 Graphical Dashboard (sales-dashboard.component.html)

Combines sales metrics and monthly trends using dx-chart and dx-pie-chart:

```

<div style="display: flex; gap: 20px; padding: 20px;">

  <!-- Monthly Sales Trend Chart -->
  <dx-chart [dataSource]="monthlySalesData" title="Monthly Sales Trend"
style="flex: 1;">
    <dx-series valueField="totalSales" argumentField="month" name="Sales ($)"
type="bar" color="#1db2f5"></dx-series>
    <dxo-legend [visible]="false"></dxo-legend>
  </dx-chart>

  <!-- Top Selling Items Pie Chart -->
  <dx-pie-chart [dataSource]="topItemsData" title="Top Selling Products"
style="flex: 1;">
    <dx-series argumentField="itemName" valueField="totalQuantity">
      <dxo-label [visible]="true" format="percent"></dxo-label>
    </dx-series>
  </dx-pie-chart>

</div>

```

Recommended Assessment Workflow Checklist

1. **Database Setup:** Execute the SQL script above in SQL Server to generate tables and sample data.
2. **Backend API:** Set up ASP.NET Core project with EF Core database context, upload directory static

files (app.UseStaticFiles()), and JWT Auth.

3. **Frontend Masters:** Create standard CRUD views for Customer & Item master screens with DevExtreme controls (dx-data-grid & dx-file-uploader).
 4. **Transaction Screen:** Build the Sales Entry screen using dynamic array binding for line items.
 5. **Dashboard & Analytics:** Bind dx-chart and dx-pivot-grid directly to backend summary endpoints.
-

User prompt: Where is user login and Master screens

Response: Here is the complete implementation for the **User Login (JWT Authentication)** and the **Master Screens (Customer & Item Master)**, including image upload and preview support.

1. User Login (JWT Authentication)

Backend: Authentication Controller (AuthController.cs)

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IConfiguration _config;
    private readonly ApplicationDbContext _context;

    public AuthController(IConfiguration config, ApplicationDbContext context)
    {
        _config = config;
        _context = context;
    }

    [HttpPost("login")]
    public IActionResult Login([FromBody] LoginDto login)
    {
        // Replace with password hashing check (e.g., BCrypt) in production
        var user = _context.Users.FirstOrDefault(u => u.Username ==
login.Username && u.PasswordHash == login.Password);
        if (user == null)
            return Unauthorized(new { message = "Invalid username or password"
});

        var token = GenerateJwtToken(user);
        return Ok(new { token, username = user.Username, fullName =
user.FullName });
    }

    private string GenerateJwtToken(User user)
```

```

    {
        var securityKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"] ??
"YourSuperSecretKey1234567890!"));
        var credentials = new SigningCredentials(securityKey,
SecurityAlgorithms.HmacSha256);

        var claims = new[]
        {
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new Claim(ClaimTypes.Name, user.Username),
            new Claim(ClaimTypes.Role, user.Role)
        };

        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience: _config["Jwt:Audience"],
            claims: claims,
            expires: DateTime.UtcNow.AddHours(8),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}

public record LoginDto(string Username, string Password);

```

Frontend: Login Component (login.component.html)

```

<div style="max-width: 400px; margin: 80px auto; padding: 30px; border: 1px
solid #ddd; border-radius: 8px;">
    <h2 style="text-align: center; margin-bottom: 20px;">Sales Management System
Login</h2>

    <dx-form [formData]="loginData">
        <dx-item dataField="username">
            <dx-validation-rule type="required" message="Username is
required"></dx-validation-rule>
        </dx-item>
        <dx-item dataField="password" [editorOptions]="{ mode: 'password' }">
            <dx-validation-rule type="required" message="Password is
required"></dx-validation-rule>
        </dx-item>
    </dx-form>

    <div style="margin-top: 20px;">
        <dx-button text="Sign In" type="default" [width]="'100%'"
(onClick)="onLogin()"></dx-button>
    </div>
</div>

```

Frontend: Login Component Logic (login.component.ts)

```
import { Component } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html'
})
export class LoginComponent {
  loginData = { username: '', password: '' };

  constructor(private http: HttpClient, private router: Router) {}

  onLogin(): void {
    this.http.post<any>('http://localhost:7000/api/auth/login', this.loginData)
      .subscribe({
        next: (res) => {
          localStorage.setItem('token', res.token);
          this.router.navigate(['/sales-entry']);
        },
        error: (err) => alert(err.error?.message || 'Login failed')
      });
  }
}
```

2. Customer Master Screen

Frontend: (customer-master.component.html)

```
<div style="padding: 20px;">
  <h2>Customer Master</h2>

  <dx-data-grid
    [dataSource]="customerDataSource"
    [showBorders]="true"
    [remoteOperations]="true"
    [rowAlternationEnabled]="true">

    <dxo-search-panel [visible]="true" placeholder="Search
customer..."></dxo-search-panel>
    <dxo-paging [pageSize]="10"></dxo-paging>

    <dxo-editing
      mode="popup"
      [allowAdding]="true"
      [allowUpdating]="true"
      [allowDeleting]="true"
```

```

        [useIcons]="true">
        <dxo-popup title="Customer Information" [showTitle]="true" [width]="600"
[height]="400"></dxo-popup>
        </dxo-editing>

        <dx-column dataField="id" caption="ID" [allowEditing]="false"
[width]="70"></dx-column>

        <dx-column dataField="customerCode" caption="Customer Code">
        <dx-validation-rule type="required" message="Code is
required"></dx-validation-rule>
        </dx-column>

        <dx-column dataField="name" caption="Customer Name">
        <dx-validation-rule type="required" message="Name is
required"></dx-validation-rule>
        </dx-column>

        <dx-column dataField="email" caption="Email"></dx-column>
        <dx-column dataField="phone" caption="Phone"></dx-column>
        <dx-column dataField="address" caption="Address"></dx-column>
    </dx-data-grid>
</div>

```

3. Item Master Screen (Image Upload & Preview)

Frontend: (item-master.component.html)

```

<div style="padding: 20px;">
    <h2>Item Master</h2>

    <dx-data-grid
        [dataSource]="itemDataSource"
        [showBorders]="true"
        [remoteOperations]="true"
        [rowAlternationEnabled]="true">

        <dxo-search-panel [visible]="true" placeholder="Search
items..."></dxo-search-panel>
        <dxo-paging [pageSize]="10"></dxo-paging>

        <dxo-editing
            mode="popup"
            [allowAdding]="true"
            [allowUpdating]="true"
            [allowDeleting]="true"
            [useIcons]="true">
            <dxo-popup title="Item Details" [showTitle]="true" [width]="700"
[height]="550"></dxo-popup>
            <dxo-form>
                <dx-item dataField="itemCode"></dx-item>

```

```

        <dx-item dataField="name"></dx-item>
        <dx-item dataField="price"></dx-item>
        <dx-item dataField="stockQuantity"></dx-item>
        <dx-item dataField="imagePath"
template="imageUploadTemplate"></dx-item>
    </dxo-form>
</dxo-editing>

    <dx-column dataField="id" caption="ID" [allowEditing]="false"
[width]="70"></dx-column>

    <!-- Image Preview Column in Grid -->
    <dx-column dataField="imagePath" caption="Preview"
cellTemplate="gridImageTemplate" [width]="100">
    </dx-column>
    <div *dxTemplate="let cell of 'gridImageTemplate'">
        <img [src]='http://localhost:7000' + cell.value" *ngIf="cell.value"
style="height: 40px; border-radius: 4px;" />
    </div>

    <dx-column dataField="itemCode" caption="Item Code">
        <dx-validation-rule type="required"></dx-validation-rule>
    </dx-column>

    <dx-column dataField="name" caption="Item Name">
        <dx-validation-rule type="required"></dx-validation-rule>
    </dx-column>

    <dx-column dataField="price" caption="Price" dataType="number"
format="$#,##0.00"></dx-column>
    <dx-column dataField="stockQuantity" caption="Stock Qty"
dataType="number"></dx-column>

    <!-- Popup Image Upload Form Item -->
    <div *dxTemplate="let data of 'imageUploadTemplate'">
        <label style="display: block; margin-bottom: 5px;">Item Image</label>
        <dx-file-uploader
            selectButtonText="Upload Image"
            labelText="or drop image here"
            accept="image/*"
            uploadMode="instantly"
            uploadUrl="http://localhost:7000/api/items/upload-image"
            (onUploaded)="onImageUploaded($event, data)">
        </dx-file-uploader>

        <div *ngIf="data.component.option('formData').imagePath"
style="margin-top: 10px;">
            <img [src]='http://localhost:7000' +
data.component.option('formData').imagePath" style="height: 90px;
border-radius: 4px; border: 1px solid #ccc;" />
        </div>
    </div>
</div>

```

```
</dx-data-grid>
</div>
```

Frontend: Item Master Component Logic (item-master.component.ts)

```
import { Component } from '@angular/core';
import CustomStore from 'devextreme/data/custom_store';
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-item-master',
  templateUrl: './item-master.component.html'
})
export class ItemMasterComponent {
  itemDataSource: CustomStore;

  constructor(private http: HttpClient) {
    this.itemDataSource = new CustomStore({
      key: 'id',
      load: () =>
this.http.get<any[]>('http://localhost:7000/api/items').toPromise(),
      insert: (values) => this.http.post('http://localhost:7000/api/items',
values).toPromise(),
      update: (key, values) =>
this.http.put(`http://localhost:7000/api/items/${key}`, values).toPromise(),
      remove: (key) =>
this.http.delete(`http://localhost:7000/api/items/${key}`).toPromise()
    });
  }

  onImageUploaded(e: any, cellInfo: any): void {
    const response = JSON.parse(e.request.responseText);
    // Assign uploaded image path to the popup form data model
    cellInfo.component.option('formData').imagePath = response.imagePath;
  }
}
```

User prompt: Please include the sales header and sales details in db schema Then in sales invoice entry screen, navigation bar at top, and updation bar at bottom Then use interactive data grid using devextreme for reporting, and kpi cards, charts for dashboard. Please include these too

Response: Here is the updated implementation incorporating the explicit Sales Header/Details relational structure, the top/bottom action layout for invoice entry, interactive reporting grids, and KPI cards with charts.

1. Database Schema (SQL Server)

```

-- 1. Users Table
CREATE TABLE Users (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Username NVARCHAR(50) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    FullName NVARCHAR(100) NOT NULL,
    Role NVARCHAR(20) NOT NULL DEFAULT 'User'
);

-- 2. Items Table
CREATE TABLE Items (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    ItemCode NVARCHAR(20) NOT NULL UNIQUE,
    Name NVARCHAR(100) NOT NULL,
    Price DECIMAL(18,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    ImagePath NVARCHAR(255) NULL
);

-- 3. Customers Table
CREATE TABLE Customers (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    CustomerCode NVARCHAR(20) NOT NULL UNIQUE,
    Name NVARCHAR(100) NOT NULL,
    Email NVARCHAR(100) NULL,
    Phone NVARCHAR(20) NULL,
    Address NVARCHAR(250) NULL
);

-- 4. Sales Header Table
CREATE TABLE SalesHeaders (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    InvoiceNumber NVARCHAR(30) NOT NULL UNIQUE,
    InvoiceDate DATETIME NOT NULL DEFAULT GETDATE(),
    CustomerId INT NOT NULL,
    SubTotal DECIMAL(18,2) NOT NULL,
    TaxAmount DECIMAL(18,2) NOT NULL DEFAULT 0.00,
    GrandTotal DECIMAL(18,2) NOT NULL,
    Notes NVARCHAR(500) NULL,
    CreatedBy INT NOT NULL,
    CONSTRAINT FK_SalesHeaders_Customers FOREIGN KEY (CustomerId) REFERENCES
Customers(Id),
    CONSTRAINT FK_SalesHeaders_Users FOREIGN KEY (CreatedBy) REFERENCES
Users(Id)
);

-- 5. Sales Details Table
CREATE TABLE SalesDetails (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    SalesHeaderId INT NOT NULL,
    ItemId INT NOT NULL,
    UnitPrice DECIMAL(18,2) NOT NULL,

```

```

Quantity INT NOT NULL,
Discount DECIMAL(18,2) NOT NULL DEFAULT 0.00,
LineTotal AS ((UnitPrice * Quantity) - Discount),
CONSTRAINT FK_SalesDetails_SalesHeaders FOREIGN KEY (SalesHeaderId)
REFERENCES SalesHeaders(Id) ON DELETE CASCADE,
CONSTRAINT FK_SalesDetails_Items FOREIGN KEY (ItemId) REFERENCES Items(Id)
);

```

2. Sales Invoice Entry Screen (Top Navigation Bar + Bottom Update Bar)

This screen layout features a top action bar for record navigation/actions, a header form, a DevExpress DataGrid for line details, and a bottom persistent status/update bar.

Template (sales-entry.component.html)

```

<div style="padding: 20px; max-width: 1200px; margin: 0 auto;">

  <!-- TOP NAVIGATION / ACTION BAR -->
  <div style="display: flex; justify-content: space-between; align-items:
center; background: #f5f5f5; padding: 10px 20px; border-radius: 6px;
margin-bottom: 20px; border: 1px solid #ddd;">
    <div style="display: flex; gap: 10px; align-items: center;">
      <dx-button icon="first" (onClick)="navigateRecord('first')" hint="First
Record"></dx-button>
      <dx-button icon="chevronleft" (onClick)="navigateRecord('prev')"
hint="Previous Record"></dx-button>
      <span style="font-weight: bold;">Invoice: {{ currentInvoiceNumber ||
'NEW' }}</span>
      <dx-button icon="chevronright" (onClick)="navigateRecord('next')"
hint="Next Record"></dx-button>
      <dx-button icon="last" (onClick)="navigateRecord('last')" hint="Last
Record"></dx-button>
    </div>

    <div style="display: flex; gap: 10px;">
      <dx-button text="New Invoice" icon="add" type="default"
(onClick)="resetForm()"></dx-button>
      <dx-button text="Print" icon="print"
(onClick)="printInvoice()"></dx-button>
    </div>
  </div>

  <!-- SALES HEADER FORM -->
  <div style="background: #fff; padding: 20px; border-radius: 6px; border: 1px
solid #e0e0e0; margin-bottom: 20px;">
    <h3>Sales Header</h3>
    <dx-form [formData]="headerData" [colCount]="3">
      <dx-item dataField="invoiceNumber" [editorOptions]="{ disabled: true
}"></dx-item>

```

```

        <dx-item dataField="customerId" editorType="dxSelectBox"
            [editorOptions]="{ dataSource: customers, displayExpr: 'name',
valueExpr: 'id', searchEnabled: true, placeholder: 'Select Customer...' }">
            <dx-validation-rule type="required" message="Customer is
required"></dx-validation-rule>
        </dx-item>

        <dx-item dataField="invoiceDate" editorType="dxDateBox"
[editorOptions]="{ width: '100%' }">
            <dx-validation-rule type="required"></dx-validation-rule>
        </dx-item>

        <dx-item dataField="notes" [colSpan]="3" editorType="dxTextArea"
[editorOptions]="{ height: 50 }"></dx-item>
    </dx-form>
</div>

<!-- SALES DETAILS GRID -->
<div style="background: #fff; padding: 20px; border-radius: 6px; border: 1px
solid #e0e0e0;">
    <h3>Sales Line Items</h3>
    <dx-data-grid
        [dataSource]="lineDetails"
        [showBorders]="true"
        (onRowInserted)="calculateTotals()"
        (onRowUpdated)="calculateTotals()"
        (onRowRemoved)="calculateTotals()">

        <dxo-editing mode="cell" [allowAdding]="true" [allowUpdating]="true"
[allowDeleting]="true"></dxo-editing>

        <dx-column dataField="itemId" caption="Item" editorType="dxSelectBox"
            [editorOptions]="{ dataSource: items, displayExpr: 'name', valueExpr:
'id', onValueChanged: onItemChanged.bind(this) }">
            <dx-validation-rule type="required"></dx-validation-rule>
        </dx-column>

        <dx-column dataField="unitPrice" caption="Unit Price" dataType="number"
format="$#,##0.00"></dx-column>
        <dx-column dataField="quantity" caption="Quantity" dataType="number">
            <dx-validation-rule type="range" [min]="1" message="Qty must be at
least 1"></dx-validation-rule>
        </dx-column>
        <dx-column dataField="discount" caption="Discount" dataType="number"
format="$#,##0.00"></dx-column>
        <dx-column caption="Line Total"
[calculateCellValue]="calculateLineTotal" format="$#,##0.00"
[allowEditing]="false"></dx-column>
    </dx-data-grid>
</div>

<!-- BOTTOM UPDATION / ACTION BAR -->

```

```

<div style="position: sticky; bottom: 0; background: #2a2d34; color: #fff;
padding: 15px 25px; border-radius: 6px 6px 0 0; margin-top: 20px; display:
flex; justify-content: space-between; align-items: center; box-shadow: 0 -2px
10px rgba(0,0,0,0.15);">
  <div style="display: flex; gap: 30px; align-items: center;">
    <div>SubTotal: <strong>{{ headerData.subTotal | currency
  }}</strong></div>
    <div>Tax (5%): <strong>{{ headerData.taxAmount | currency
  }}</strong></div>
    <div style="font-size: 1.2rem; color: #4caf50;">Grand Total: <strong>{{
headerData.grandTotal | currency }}</strong></div>
  </div>

  <div style="display: flex; gap: 12px;">
    <dx-button text="Cancel" type="normal"
(onClick)="resetForm()"></dx-button>
    <dx-button text="Save / Update Invoice" type="success" icon="save"
(onClick)="saveInvoice()"></dx-button>
  </div>
</div>
</div>

```

3. Reporting & Dashboard

3.1 Interactive Sales Report Grid (sales-report.component.html)

Interactive DevExtreme DataGrid featuring column grouping, summary footers, date range filtering, and Excel export support.

```

<div style="padding: 20px;">
  <h2>Interactive Sales Report</h2>

  <dx-data-grid
    [dataSource]="reportDataSource"
    [showBorders]="true"
    [columnAutoWidth]="true"
    [allowColumnReordering]="true"
    [allowColumnResizing]="true">

    <dxo-search-panel [visible]="true" placeholder="Search
sales..."></dxo-search-panel>
    <dxo-header-filter [visible]="true"></dxo-header-filter>
    <dxo-group-panel [visible]="true" emptyPanelText="Drag a column header here
to group by that column"></dxo-group-panel>
    <dxo-export [enabled]="true" fileName="Sales_Report"></dxo-export>
    <dxo-paging [pageSize]="15"></dxo-paging>

    <dx-column dataField="invoiceNumber" caption="Invoice #"></dx-column>
    <dx-column dataField="invoiceDate" caption="Date" dataType="date"

```

```

format="yyyy-MM-dd"></dx-column>
    <dx-column dataField="customerName" caption="Customer"
[groupIndex]="0"></dx-column>
    <dx-column dataField="itemName" caption="Item"></dx-column>
    <dx-column dataField="quantity" caption="Qty"
dataType="number"></dx-column>
    <dx-column dataField="unitPrice" caption="Unit Price"
format="$#,##0.00"></dx-column>
    <dx-column dataField="grandTotal" caption="Total Amount"
format="$#,##0.00"></dx-column>

    <!-- Group & Total Summaries -->
    <dxo-summary>
        <dx-group-item column="grandTotal" summaryType="sum"
valueFormat="$#,##0.00" displayFormat="Total: {0}"></dx-group-item>
        <dx-total-item column="grandTotal" summaryType="sum"
valueFormat="$#,##0.00" displayFormat="Grand Total: {0}"></dx-total-item>
    </dxo-summary>

</dx-data-grid>
</div>

```

3.2 Graphical Dashboard with KPI Cards (dashboard.component.html)

Combines top KPI cards (Total Revenue, Total Invoices, Total Customers) with interactive DevExtreme charts:

```

<div style="padding: 20px; background: #f8f9fa;">
    <h2>Executive Sales Dashboard</h2>

    <!-- 1. KPI CARDS -->
    <div style="display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px;
margin-bottom: 25px;">

        <div style="background: #fff; padding: 20px; border-radius: 8px;
border-left: 5px solid #28a745; box-shadow: 0 2px 5px rgba(0,0,0,0.05);">
            <div style="color: #6c757d; font-size: 0.9rem;">Total Revenue</div>
            <div style="font-size: 1.8rem; font-weight: bold; margin-top: 5px;">{{
kpiData.totalRevenue | currency }}</div>
        </div>

        <div style="background: #fff; padding: 20px; border-radius: 8px;
border-left: 5px solid #17a2b8; box-shadow: 0 2px 5px rgba(0,0,0,0.05);">
            <div style="color: #6c757d; font-size: 0.9rem;">Invoices Generated</div>
            <div style="font-size: 1.8rem; font-weight: bold; margin-top: 5px;">{{
kpiData.totalInvoices }}</div>
        </div>

        <div style="background: #fff; padding: 20px; border-radius: 8px;
border-left: 5px solid #ffc107; box-shadow: 0 2px 5px rgba(0,0,0,0.05);">

```

```
        <div style="color: #6c757d; font-size: 0.9rem;">Active Customers</div>
        <div style="font-size: 1.8rem; font-weight: bold; margin-top: 5px;">{{
kpiData.activeCustomers }}</div>
    </div>

</div>

<!-- 2. CHARTS SECTION -->
<div style="display: grid; grid-template-columns: 2fr 1fr; gap: 20px;">

    <!-- Monthly Revenue Bar Chart -->
    <div style="background: #fff; padding: 20px; border-radius: 8px;
box-shadow: 0 2px 5px rgba(0,0,0,0.05);">
        <dx-chart [dataSource]="monthlySales" title="Monthly Revenue Analysis">
            <dx-series valueField="revenue" argumentField="month" name="Revenue"
type="bar" color="#007bff"></dx-series>
            <dxo-tooltip [enabled]="true" format="currency"></dxo-tooltip>
            <dxo-legend [visible]="false"></dxo-legend>
        </dx-chart>
    </div>

    <!-- Product Revenue Share Pie Chart -->
    <div style="background: #fff; padding: 20px; border-radius: 8px;
box-shadow: 0 2px 5px rgba(0,0,0,0.05);">
        <dx-pie-chart [dataSource]="topProducts" title="Top Products Share">
            <dx-series argumentField="productName" valueField="totalSales">
                <dxo-label [visible]="true" format="percent"></dxo-label>
            </dx-series>
            <dxo-tooltip [enabled]="true"></dxo-tooltip>
        </dx-pie-chart>
    </div>

</div>
</div>
```
